SOFTWARE STUDIO · DESIGN PATTERNS

Alpha Draft

A fantasy league for stock picking, and the Strategy and Decorator patterns in our own code

Hudson Plattus · Justin Saphier

OVERVIEW

Two patterns, both at work when a stock is saved

PATTERN 1 · SCRUM-23

Strategy for stock validation rules

`StockValidationRule`

PATTERN 2 · STOCK POOL AUDIT

Decorator that records every change to the pool

`AuditingStockRepository`

Strategy decides whether a stock may be saved. Decorator records every save, without changing the code that saves it.

PATTERN 1 · THE PROBLEM

The backend accepted any stock it was sent

StockService had no validation: a blank ticker, a made-up sector or a duplicate ticker all went through. We needed several independent checks, with more to come as the league grows. The obvious way to add them is inline, like this:

```
// The approach we avoided: every rule written inline in StockService
if (stock.ticker() == null || stock.ticker().isBlank()) {
    errors.put("ticker", "Ticker is required.");
}
if (!SECTORS.contains(stock.sector())) { errors.put("sector", …); }
// …and another block in StockService for every rule the league adds
```

PATTERN 1 · WHY STRATEGY

Why Strategy fit better than the alternatives

Option	What it does	Fit for validation rules?
Strategy	Each rule is a class behind one interface	Yes. Every rule runs and reports; a new rule is a new class.
Chain of Responsibility	Passes a request along until one handler takes it	No. It stops at the first handler; we want every error at once.
Template Method	Fixes the steps; subclasses fill some in	No. The rules are independent checks, not steps of one algorithm.
if/else in StockService	Checks each rule inline	Works, but every new rule edits StockService (breaks Open/Closed).

PATTERN 1 · THE FIX IN CODE

One interface, one class per rule

```
// StockValidationRule.java: every rule has the same shape
public interface StockValidationRule {
    Map<String, String> validate(StockRequest stock, Integer idBeingUpdated);
}
// StockService.java: runs every rule Spring injects, without knowing what any rule checks
for (StockValidationRule rule : validationRules) {
    rule.validate(stock, idBeingUpdated).forEach(errors::putIfAbsent);
}
if (!errors.isEmpty()) { throw new StockValidationException(errors); }
```

`RequiredFieldsRule`

Ticker, name and sector must be filled in

`AllowedSectorRule`

Sector must be one of the 11 GICS sectors

`UniqueTickerRule`

No other stock already uses this ticker

PATTERN 2 · THE PROBLEM

Nothing recorded changes to the stock pool

Every player drafts from one shared pool of stocks, and admins add, edit and delete them. The app kept no record of what changed or when, and our README already flags audit logging as an open decision. The obvious fix is to add logging inline:

```
// The approach we avoided: audit code mixed into every StockService method
Stock saved = stockRepository.save(stock);
auditLog.add(new StockAuditEntry("CREATE", saved.getId(), saved.getTicker(), Instant.now()));
// …repeated in updateStock and deleteStock, and in any new method that changes stocks
```

PATTERN 2 · WHY DECORATOR

Why Decorator fit better than the alternatives

Option	What it does	Fit for auditing changes?
Decorator	Wraps an object; same interface, added behavior	Yes. Every save and delete passes through; nothing else changes.
Adapter	Converts one interface into another	No. The interface stays exactly the same.
Subclass the repository	Overrides save and delete	Ties auditing to InMemory; a database repository would need its own copy.
Log in StockService	Adds audit lines to each method	Gives StockService a second job; easy to forget in new methods.

PATTERN 2 · THE FIX IN CODE

A repository that wraps a repository

```
// AuditingStockRepository.java: implements StockRepository AND wraps one
public class AuditingStockRepository implements StockRepository {
    private final StockRepository delegate;
    public Stock save(Stock stock) {
        String action = delegate.findById(stock.getId()).isPresent() ? "UPDATE" : "CREATE";
        Stock saved = delegate.save(stock);    // the real work, unchanged
        record(action, saved.getId(), saved.getTicker());  // the added behavior
        return saved;
    }
}
```

```
// RepositoryConfig.java: the decoration happens in one line
return new AuditingStockRepository(new InMemoryStockRepository());
```

HOW THEY CONNECT

One new stock, end to end



A new rule is a new class. New behavior on saves is a new wrapper. StockService changes for neither.